MOD -03 - JDK new Features

MOD -03 - JDK new Features

LUMINAR
TECHNOLAB

Sealed Classes (Finalized in JDK 17)

Used to control which classes can extend a class or implement an interface.

sealed abstract is common

a sealed class does NOT have to be abstract

In Java sealed classes, each permitted subclass must be exactly one of these three:

1. *final*
2. *sealed*
3. *non-sealed*

```java
package jdk17pack;


sealed abstract class Payment permits CreditCard, UPI, NetBanking {

    // Concrete method (common for all)
    public void printReceipt(double amount) {
        System.out.println("Payment of ₹" + amount + " processed successfully.");
    }


    // Abstract method (must be implemented)
    public abstract void pay(double amount);
    }
```

```java
final class CreditCard extends Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using Credit Card");
        printReceipt(amount); // calling concrete method
    }
}

sealed class UPI extends Payment permits Mobikwik { // partially open
    @Override
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using UPI");
        printReceipt(amount);
    }
}
```

```java
final class CreditCard extends Payment { //closed - way 1
    @Override
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using Credit Card");
        printReceipt(amount); // calling concrete method
    }
}

sealed class UPI extends Payment permits Mobikwik { // partially open - way 2
    @Override
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using UPI");
        printReceipt(amount);
    }
}
final class Mobikwik extends UPI {
    /*
    * public void pay(double amount) {
    * System.out.println("Paid ₹" + amount + " using UPI");
    * printReceipt(amount);
    * }
    */
}
```

A subclass doesn't need to implement an abstract method if it has already been implemented in its parent class.

```java
non-sealed class NetBanking extends Payment { // open - way 3

    @Override
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using Net Banking");
        printReceipt(amount);
    }
}
```

```java
// not allowed . eg.1



class DebitCard extends Payment {
    @Override
    public void pay(double amount) {

    }
}


// not allowed . eg.2


final class DebitCard extends Payment {
}
```

The class DebitCard with a sealed direct superclass or a sealed direct superinterface Payment should be declared either final, sealed, or non-sealed

DebitCard is not in permits list => Compile-time error

// not allowed . eg.3

If a subclass is also sealed, it must define its own permits list (eg. permits PhonePeWallet)
ie. sealed class PhonePe extends Payment permits PhonePeWallet

The type PhonePe extending a sealed class Payment should be a permitted subtype of Payment.
ie. sealed abstract class Payment permits CreditCard, UPI, NetBanking,PhonePe

```
sealed class PhonePe extends Payment permits PhonePeWallet {
    @Override
    public void pay(double amount) {

    }
}

final class PhonePeWallet extends PhonePe {
}
```

```java
public class SealedClassDemo {
    public static void main(String[] args) {
        Payment upi = new UPI();
        upi.pay(1500);
    }
    }
```

Pattern Matching for instanceof

Before Java 17, you had to:

- Check using instanceof
- Cast manually again
- Then use the variable

Java 17 removes unnecessary casting.

Program 1

```java
package jdk17pack;

public class TestPatternMatching {

    public static void main(String[] args) {
        Object obj = "Hello Java 17"; // Object containing String

        System.out.println("=== BEFORE Java 17 ===");

        if (obj instanceof String) {
            String s = (String) obj; // manual cast
            System.out.println("Uppercase: " + s.toUpperCase());
            System.out.println("Length: " + s.length());
        }

        System.out.println("\n=== AFTER Java 17 (Pattern Matching) ===");

        if (obj instanceof String s) { // pattern variable created
            System.out.println("Uppercase: " + s.toUpperCase());
            System.out.println("Length: " + s.length());
        }
    }
}
```

Program 2

```java
package jdk17pack.testmatch;


public class Admin {
String name = "Admin User";


void accessDashboard() {
System.out.println("Accessing Admin Dashboard");
}
}
```

```java
package jdk17pack.testmatch;


public class Customer {
String name = "Customer User";


void viewOrders() {
System.out.println("Viewing Orders");
}
}
```

```java
package jdk17pack.testmatch;


public class Guest {
String name = "Guest User";


void browse() {
System.out.println("Browsing Products");
}
}
```

```java
package jdk17pack.testmatch;

public class TestPatternMatchingDemo {
    public static void main(String[] args) {

        Object user = new Customer(); // this is in a class

        if (user instanceof Admin admin) { // passed this value to another class
            System.out.println(admin.name);
            admin.accessDashboard();

        } else if(user instanceof Customer customer) {//passed this value to another class
            System.out.println(customer.name);
            customer.viewOrders();

        } else if (user instanceof Guest guest) {// passed this value to another class
            System.out.println(guest.name);
            guest.browse();
        }
    }
}
```

Modified in Enhanced Switch Expression

Switch can now RETURN a value

Before Java 17 -> switch could not return a value directly.

Now -> switch is an expression, not just a statement.

Old switch used colon + break

New switch uses arrow (->) which is more modern and readable.

```java
package jdk17pack;

public class TestEnhancedSwitch {
public static void main(String[] args) {

int day = 3;

String message = switch (day) {
case 1 -> "Monday";
case 2 -> "Tuesday";
case 3 -> "Wednesday";
case 4 -> "Thursday";
case 5 -> "Friday";
case 6 -> "Saturday";
case 7 -> "Sunday";
default -> "Invalid Day!";
};

System.out.println("Today is: " + message);
}
}
```

Before (old switch)

```java
String dayName;
switch (day) {
   case 1:
      dayName = "Monday";
      break;
   default:
      dayName = "Unknown";
}
```

After (enhanced switch)

```java
String dayName = switch (day) {
   case 1 -> "Monday";
   default -> "Unknown";
};
```

Records

A record is a special kind of class in Java designed to hold data. It automatically provides:

- Constructor
- Getters
- equals()
- hashCode()
- toString()

You don’t write any boilerplate code.

Without Record (Old Java)

```
class Student {
    private final String name;
    private final int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    @Override
    public String toString() {
        return "Student{name='" + name + "', age=" + age + "}";
    }
}
```

With Record (JDK 17)

```
public record Student(String name, int age) { }
```

That’s all:

Java automatically generates all the methods behind the scenes.

```java
package jdk17pack;

record Student(String name, int age) { }


public class TestRecord {


    public static void main(String[] args) {
        Student student = new Student("Bini", 24);
        // student.name="arun";//the field Student.name is not visible


        System.out.println(student.name()); // getter
        System.out.println(student.age());
        System.out.println(student); // auto toString()
    }
}
```

Records are immutable

record, the fields are implicitly private and final.-> you cannot change them.

Student.name="Arun"; ❌

Text Blocks

A text block is simply a multi-line string written using triple quotes """.

It improves readability especially for:

- JSON
- SQL Queries
- HTML
- XML

Multi-line messages , JavaScript code in Java

Before Text Blocks

```
String query =
        "SELECT emp_id, emp_name, emp_role,
emp_salary\n" +
        "FROM employee_table\n" +
        "WHERE emp_role = 'DEVELOPER'\n" +
        "AND emp_salary > 60000\n" +
        "ORDER BY emp_salary DESC;";
```

AfterText Blocks

```
String query = """
        SELECT emp_id, emp_name, emp_role,
emp_salary
        FROM employee_table
        WHERE emp_role = 'DEVELOPER'
        AND emp_salary > 60000
        ORDER BY emp_salary DESC;
        """;
```

Thank you . Happy Learning ☺